



CLASSES DE COMPORTAMENTOS

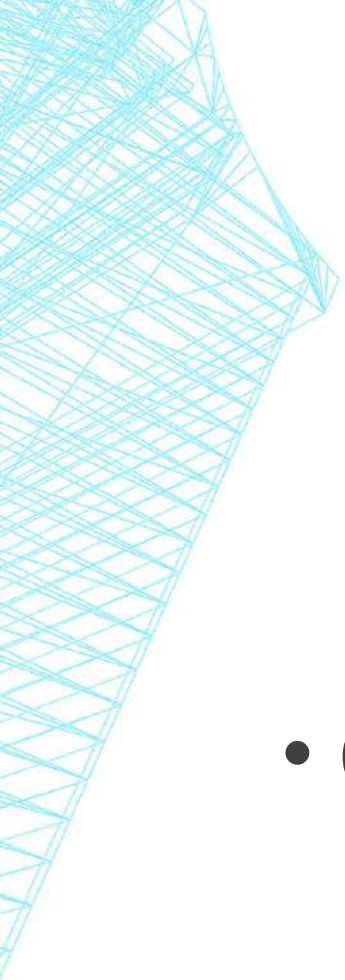
CLASSES DE COMPORTAMENTO

- Seja $f(n)$ a função de complexidade de um algoritmo.
- Então, dizemos que $O(f(n))$ é complexidade assintótica do algoritmo.
- Problemas com comportamento similar podem ter a mesma complexidade assintótica:
 - Classes de comportamento assintótico



CLASSES DE COMPORTAMENTO

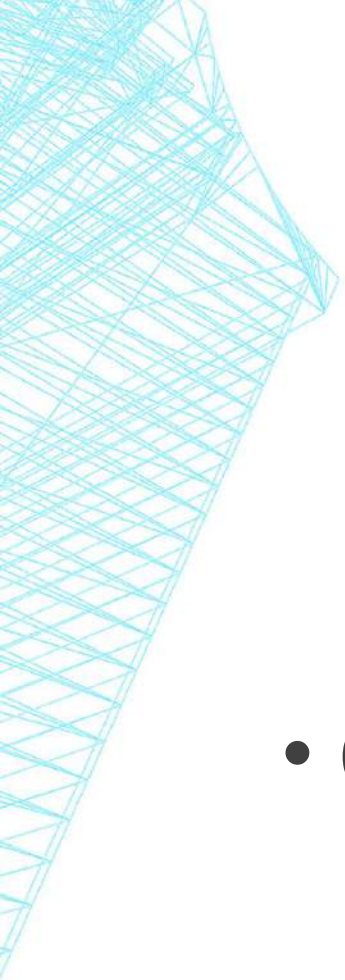
- Algoritmos da mesma classe de comportamento apresentam características típicas.
 - Porém, atenção para as exceções.



COMPLEXIDADE FIXA OU CONSTANTE

$$f(n) = O(1)$$

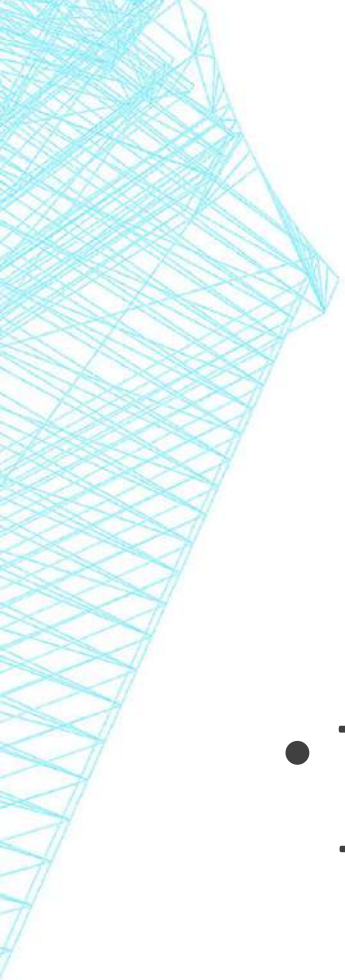
- O custo do algoritmo independe do tamanho de n .
- As instruções do algoritmo são executadas um número fixo de vezes.



COMPLEXIDADE FIXA OU CONSTANTE

$$f(n) = O(c)$$

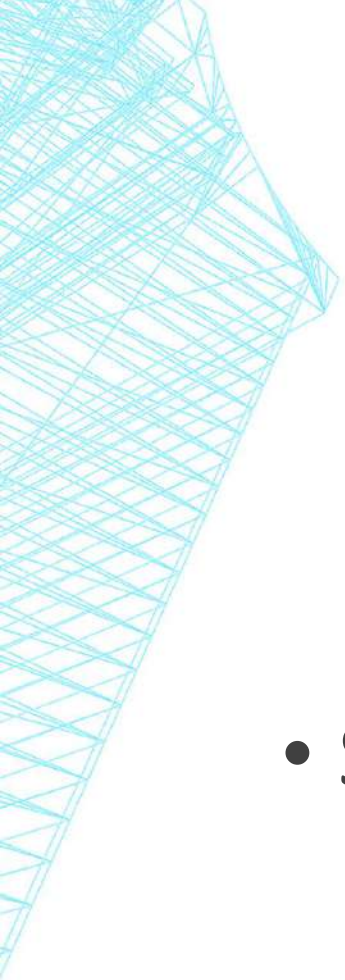
- O custo do algoritmo independe do tamanho de n .
- As instruções do algoritmo são executadas um número fixo de vezes.



COMPLEXIDADE LOGARÍTMICA

$$f(n) = O(\log n)$$

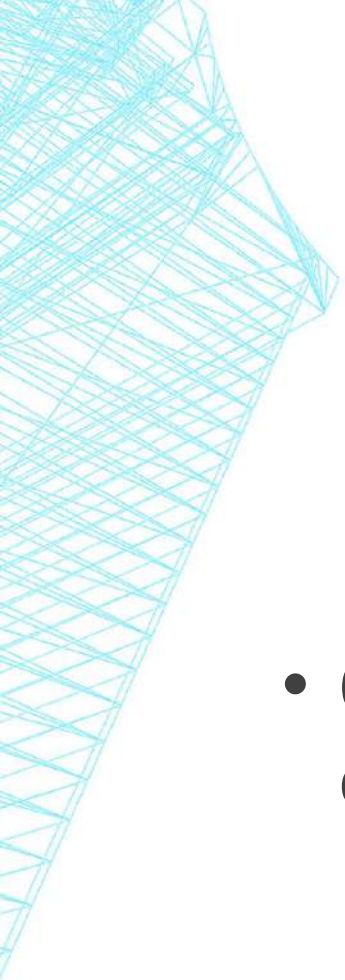
- Tipicamente algoritmos que resolvem problemas transformando-os em problemas menores.



COMPLEXIDADE LOGARÍTMICA

$$f(n) = O(\log n)$$

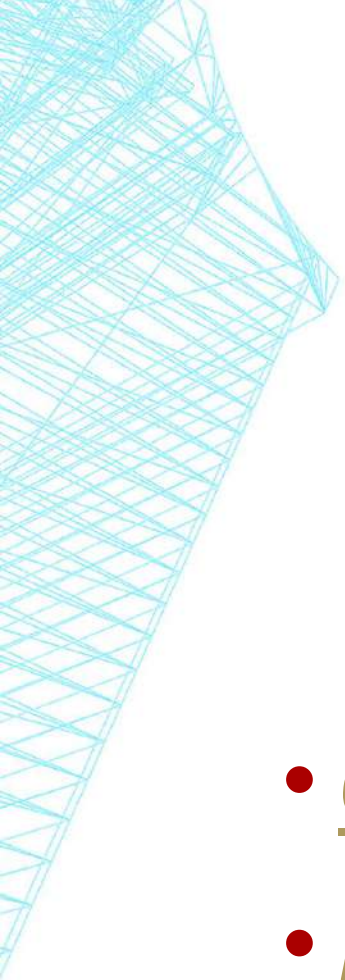
- Supondo que a base do logaritmo seja 2:
 - Para $n = 1.000$, $\log_2 \cong 10$
 - Para $n = 1.000.000$, $\log_2 \cong 20$



COMPLEXIDADE LINEAR

$$f(n) = O(n)$$

- Operações realizadas sobre cada elemento dos dados de entrada.
 - Ex: operações lineares em vetores.
- Cada vez que n dobra de tamanho, o tempo de execução também dobra.



ALGORITMOS $N \times \log N$

$$f(n) = O(n \log n)$$

- Quebram o problema em partes menores;
- Resolvem cada um deles independentemente;
- Agrupam as soluções.

ALGORITMOS $N \times \log N$

$$f(n) = O(n \log n)$$

- Supondo que a base do logaritmo seja 2:
 - Para $n = 1.000.000$, $n \log_2 n \cong 20.000.000$
 - Para $n = 2.000.000$, $n \log_2 n \cong 42.000.000$

ALGORITMOS QUADRÁTICOS

$$(n) = O(n^2)$$

▪ Ex: **n**

n²

10

100

100

10.000

1.000

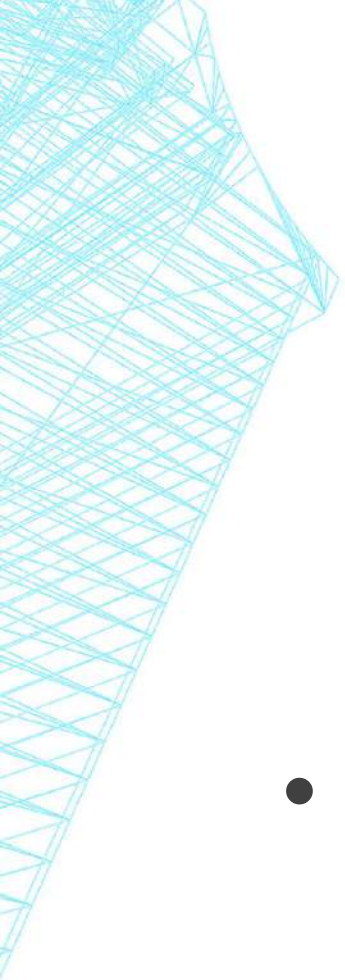
1.000.000

10.000

100.000.000

100.000

10.000.000.000



ALGORITMOS CÚBICOS

$$f(n) = O(n^3)$$

- Úteis apenas para resolver problemas relativamente pequenos.
 - Sempre que n dobra, o tempo de execução é multiplicado por 8.

ALGORITMOS CÚBICOS

$$f(n) = O(n^3)$$

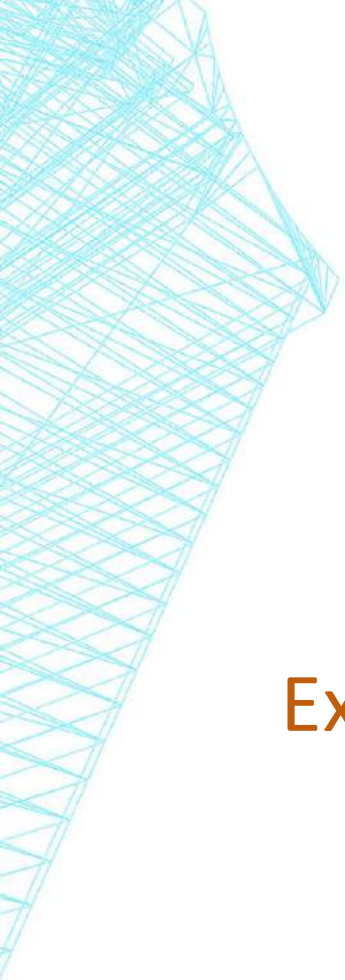
Ex:

n	n^3
10	1.000
100	1.000.000
1.000	1.000.000.000
10.000	1.000.000.000.000
100.000	100.000.000.000.000

ALGORITMOS EXPONENCIAIS

$$f(n) = O(2^n) \text{ ou } f(n) = O(c^n)$$

- Não são úteis sob o ponto de vista prático.
- Em geral, ocorrem na solução de problemas por algoritmos utilizando força bruta.
 - Sempre que n dobra, o tempo de execução fica elevado ao proporcionalmente a c .

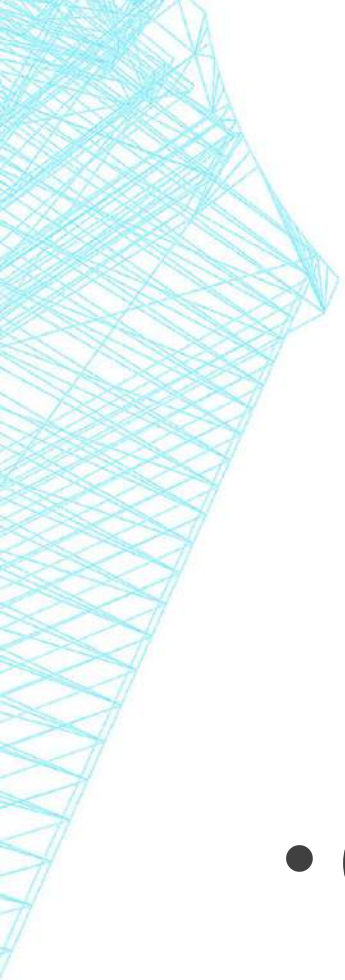


ALGORITMOS EXPONENCIAIS

$$f(n) = O(2^n)$$

Ex:

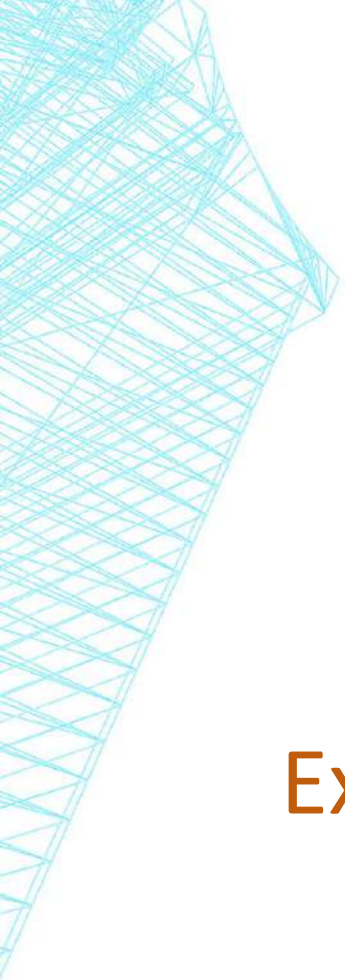
n	2^n
10	1.024
20	1.048.576
40	1.099.511.627.776
80	1.208.925.819.614.629.174.706.176



ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

- Comportamento ainda pior do que $O(2^n)$
 - Multiplicação crescente: $x2, x3, x4, x5 \dots$



ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

Ex:	n	n!
	10	3.628.800
	15	1.307.674.368.000
	20	2.432.902.008.176.640.000
	25	15.511.210.043.330.985.984.000.000

$$f(n) = O(n!)$$

- Ex: Para $n = 40$, $40! \cong$
816.000.000.000.000.000.000.000.000.000.000.000.0
00.000.000.000
- um número com **48 dígitos!**

ALGORITMOS FATORIAIS

$$f(n) = O(n!)$$

- $40! \approx 8,16 \times 10^{47}$
- Frontier: $1,685 \times 10^{18}$ instruções/segundo.
- $\frac{8,16 \times 10^{47}}{1,685 \times 10^{18}} = 4,84 \times 10^{29}$ segundos.
- = 15.356.195.999.259.289.369 milênios.

TEMPOS DE EXECUÇÃO

Um computador que execute
1 bilhão (10^9) de instruções por segundo

	10	20	30	40	50	60
n	0,00000001	0,00000002	0,00000003	0,00000004	0,00000005	0,00000006
n ²	0,0000001	0,0000004	0,0000009	0,0000016	0,0000025	0,0000036
n ³	0,000001	0,000008	0,000027	0,000064	0,000125	0,000216
n ⁵	0,0001	0,0032	0,0243	0,1024	0,3125	0,7776
2 ⁿ	0,000001024	0,001048	1,0737418	1.099,5 s = 18 min	1.125.899 s = 312 h	1.152.921.504 s = 36,5 anos
3 ⁿ	0,000059049	3,4867 s	205.891 s = 57,2 h	12.157.665.459 s = 385 anos	227.488 séculos	134 milhões de séculos